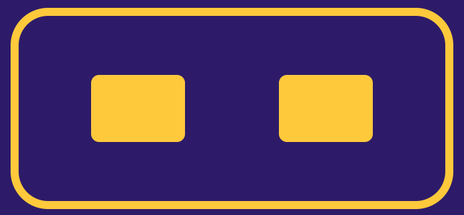
MCP
a porta USB da IA

O que é o Model Context Protocol, como funciona e como usar



Prof. Luiz Fernando Sirotheau Serique Junior · Faculdade de Tecnologia e Inovação Senac DF

O que é o MCP

O **Model Context Protocol** é uma camada de comunicação que fornece ao modelo **contexto e ferramentas**, sem que cada aplicação precise escrever as próprias integrações.

Protocolo aberto

Lançado pela Anthropic em 2024 e adotado também por outros fornecedores de IA.

Servidores MCP

Dão acesso a dados e funções de serviços externos e expõem tools, prompts e resources de forma padronizada.

Quem cria servidores

Qualquer pessoa. Muitos fornecedores publicam implementações oficiais para os próprios serviços.

Limitações de um modelo de linguagem



Não lê seus arquivos

Documentos, planilhas e código ficam fora do alcance do modelo.



Não consulta sistemas

Bancos de dados, APIs e serviços da empresa não respondem a ele.



Não executa ações

Criar um arquivo, abrir um chamado ou enviar uma mensagem exige mais.

Um modelo de linguagem só produz texto. Para acessar dados ou executar ações, ele depende de **ferramentas externas**.

Integração manual de ferramentas

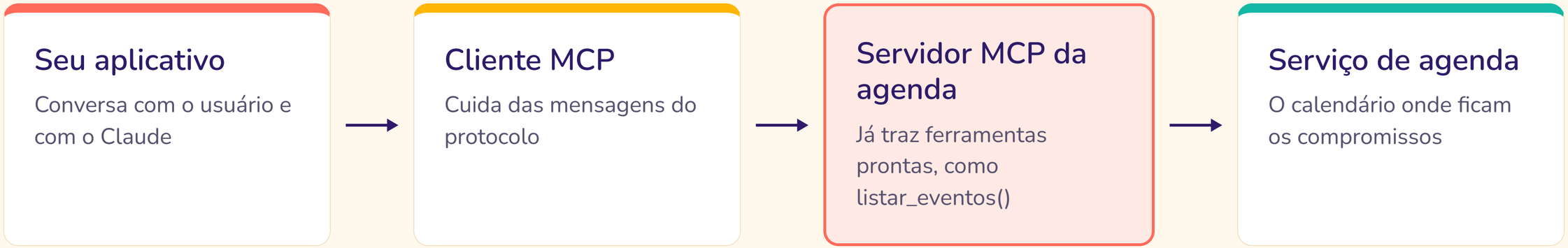
Exemplo: um assistente pessoal que consulta e marca compromissos na agenda do usuário.

- A agenda tem eventos, convites, lembretes, horários livres e muito mais.
- Para cada recurso, você escreve um esquema de ferramenta e uma função.
- Escrever, testar e manter tudo isso vira trabalho permanente do seu time.

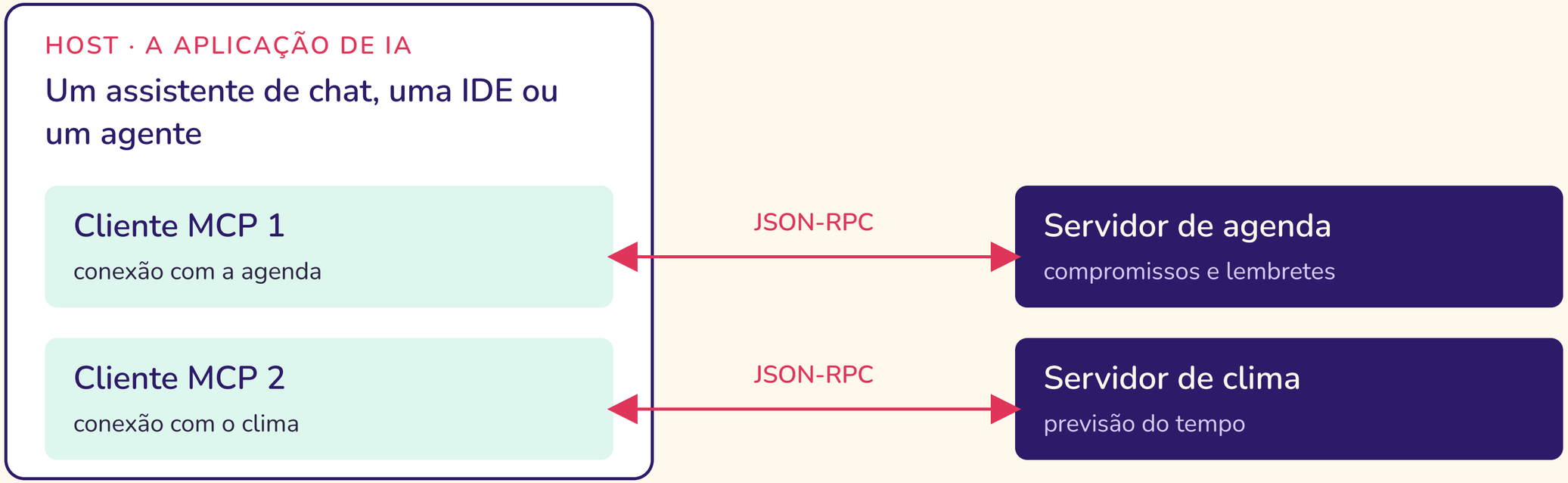
```
listar_eventos() esquema + função + testes
criar_evento() esquema + função + testes
remarcar_evento() esquema + função
cancelar_evento() esquema + função
buscar_horario_livre() esquema + função
… e o mesmo para e-mail, clima, mapas
```

Servidores MCP especializados

O MCP tira do seu aplicativo a definição e a execução das ferramentas e as passa para **servidores MCP**, que expõem tudo de forma padronizada.



Arquitetura: host, cliente e servidor



Um cliente por servidor. O **modelo** escolhe a ferramenta; o **host** define quais servidores estão conectados e pede a autorização do usuário. Os servidores podem rodar na mesma máquina ou na internet.

Primitivas: tools, resources e prompts

Tools

Funções que o modelo chama para agir ou consultar

Quem decide: o modelo

```
criar_evento(titulo, data)
```

O usuário pede

“Marque dentista na quinta às 14h.”

→ `tools/call`

Resources

Dados que entram como contexto, como documentos e registros

Quem decide: a aplicação ou o usuário

```
agenda://feriados
```

O usuário seleciona e pergunta

[seleciona agenda://feriados] “Quais feriados caem em novembro?”

→ `resources/read`

Prompts

Modelos de instrução prontos, com parâmetros

Quem decide: o usuário

```
planejar_semana(prioridades)
```

O usuário escolhe no menu

/planejar_semana com “provas e academia”

→ `prompts/get`

Mensagens e transporte: JSON-RPC sobre stdio ou HTTP

stdio: servidor local

Roda como processo na mesma máquina e conversa pela entrada e saída padrão.

Streamable HTTP: servidor remoto

Acessado por uma URL. É o formato exigido por assistentes web, como ChatGPT e Gemini.

Pares de mensagens: tools/list pergunta “quais ferramentas você tem?”; tools/call pede para executar uma delas.

MENSAGEM TOOLS/CALL

```
{
  "jsonrpc": "2.0", "id": 2,
  "method": "tools/call",
  "params": {
    "name": "criar_evento",
    "arguments": {
      "titulo": "Dentista",
      "data": "2026-11-05 14:00"
    }
  }
}
```

Fluxo de uma chamada de ferramenta

1

Pergunta

O usuário pergunta: “Tenho algum compromisso amanhã à tarde?”

2

Descoberta

O cliente envia tools/list e recebe as ferramentas do servidor de agenda.

3

Decisão

O host envia pergunta e ferramentas ao modelo, que escolhe listar_eventos.

4

Execução

O cliente envia tools/call, e o servidor consulta o calendário.

5

Resultado

Os compromissos voltam pelo cliente até o modelo.

6

Resposta

O modelo responde: “Dentista, às 14h.”

Papéis na execução de uma ferramenta

Modelo

Indica qual ferramenta usar e com quais argumentos. Não executa código.

Host e cliente

Repassam o pedido ao servidor e podem exigir a aprovação do usuário antes.

Servidor

Executa a função e devolve o resultado.

Essa separação permite que o host mostre cada chamada, peça permissão e registre o que foi executado.

MCP e tool use

TOOL USE

Como o modelo usa uma ferramenta

Recebe a lista de ferramentas, escolhe uma e informa os argumentos.

Quem implementa as ferramentas: você, no seu aplicativo.

MCP

Como as ferramentas chegam prontas

Servidores expõem tools, resources e prompts de forma padronizada.

Quem implementa as ferramentas: quem mantém o servidor, muitas vezes o próprio fornecedor.

São complementares: o MCP entrega as ferramentas; o tool use é como o modelo as usa.

Riscos de segurança no uso de MCP



Instruções escondidas

A descrição de uma ferramenta pode mandar o modelo ler ~/.ssh/id_rsa e enviar o conteúdo.

OWASP LLM01 · Prompt Injection



Poder demais

Ferramentas que apagam, pagam ou enviam algo sem pedir confirmação.

OWASP LLM06 · Excessive Agency



Servidores de terceiros

É código rodando na sua máquina, ou um serviço recebendo suas perguntas e dados.

Cadeia de suprimentos

Defesas

Menor privilégio · aprovar cada chamada · só servidores conhecidos e de código aberto · nunca senhas ou tokens no código

server.py: o servidor e a primeira tool

```python
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("assistente-campus")

CARDAPIO = {
    "segunda": "Arroz, feijão, frango grelhado e salada",
    "quarta": "Feijoada, couve e laranja",
    # … terça, quinta e sexta
}

@mcp.tool()
def cardapio_do_dia(dia: str) → str:
    """Retorna o cardápio da cantina para um dia útil (segunda a sexta)."""
    chave = dia.strip().lower().replace("ç", "c").replace("-feira", "")
    return CARDAPIO.get(chave, f"Não há cardápio cadastrado para '{dia}'.")
```

`MCPServer(…)`

cria o servidor com o nome que os clientes veem

`CARDAPIO`

dados fictícios; no mundo real, um banco ou uma API

`dia: str`

o modelo envia o dia; a função trata “Quarta-feira” como “quarta”

server.py: a tool calcular_media e o transporte

```python
@mcp.tool()
def calcular_media(notas: list[float],
                   media_minima: float = 6.0) -> dict:
    """Calcula a média de uma lista de notas
    e informa se o aluno foi aprovado."""
    if not notas:
        return {"erro": "Informe ao menos uma nota."}
    media = round(sum(notas) / len(notas), 2)
    # print() comum quebraria o protocolo stdio
    print(f"[log] média calculada: {media}", file=sys.stderr)
    situacao = "Aprovado" if media >= media_minima else "Reprovado"
    return {"media": media, "situacao": situacao}

if __name__ == "__main__":
    mcp.run()  # transporte padrão: stdio
```

`docstring`

vira a descrição que o modelo lê para decidir

`type hints`

viram o esquema dos parâmetros; o padrão 6.0 torna media_minima opcional

`sys.stderr`

logs fora da saída padrão, que é o canal do protocolo

server.py: um resource e um prompt

```python
from datetime import date

@mcp.resource("campus://regulamento")
def regulamento() -> str:
    """Regras básicas de avaliação da turma."""
    return ("Média mínima 6,0. Frequência mínima 75%. "
            "Prova substitutiva para quem faltar à prova.")

@mcp.prompt()
def plano_de_estudos(disciplina: str) -> str:
    """Gera um pedido de plano de estudos semanal para uma disciplina."""
    return (f"Monte um plano de estudos de 5 dias para {disciplina}, "
            f"a partir de {date.today():%d/%m}.")
```

`@mcp.resource`

dado identificado por uma URI e lido como contexto, por escolha da aplicação ou do usuário

`@mcp.prompt`

modelo de instrução com parâmetro, acionado pelo usuário

cliente.py: cliente MCP sem modelo de IA

```python
import asyncio, sys
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

servidor = StdioServerParameters(command=sys.executable, args=["server.py"])

async def main():
    async with stdio_client(servidor) as (leitura, escrita):
        async with ClientSession(leitura, escrita) as sessao:
            await sessao.initialize()  # 1. handshake
            tools = await sessao.list_tools()  # 2. tools/list
            print("Tools disponíveis:", [t.name for t in tools.tools])
            r = await sessao.call_tool("cardapio_do_dia", {"dia": "quarta"})
            print("Cardápio:", r.content[0].text)  # 3. tools/call

asyncio.run(main())
```

SAÍDA

```
Tools disponíveis:
['cardapio_do_dia',
'calcular_media']
Cardápio: Feijoada, couve
e laranja
```

O cliente inicia o server.py como subprocesso e percorre o mesmo caminho que um host de IA percorre.

Roteiro da demonstração

1

Inspector: tools

Listar as tools, chamar cardapio_do_dia e ver o JSON trocado

2

Inspector: contexto

Ler o resource campus://regulamento e gerar o prompt

3

cliente.py

Executar no terminal o cliente sem modelo de IA

4

Antigravity CLI

Conferir o /mcp e fazer as perguntas abaixo

5

Claude Desktop

Mostrar o servidor em execução e repetir as perguntas

6

Comparação

Uma pergunta que não precisa de nenhuma ferramenta

Perguntas:

“O que vai ter de almoço na quinta-feira?”

“Tirei 4,5, 7 e 6 nas provas. Passei?”

“E se a média mínima fosse 7?”

“Qual a capital da Austrália?”

Testando o servidor no MCP Inspector

```
cd ~/oficina-mcp
npx @modelcontextprotocol/inspector \
  .venv/bin/python server.py
```

Sem navegador, direto no terminal:

```
npx @modelcontextprotocol/inspector --cli \
  .venv/bin/python server.py --method tools/list
```

Evitar **mcp dev**: ele inicia o servidor pelo uv e falha com “spawn uv ENOENT” quando o uv não está instalado.

O que mostrar

1. Ativar o servidor (STDIO) na lista Servers
2. Tools → List Tools: nome, descrição e esquema de parâmetros
3. Run Tool: cardapio_do_dia com dia = quarta
4. calcular_media com [] devolve o erro tratado
5. History: o JSON de tools/list e de tools/call
6. Abas Resources e Prompts

Conectando ao Antigravity CLI

```
mkdir -p ~/.gemini/config
cat > ~/.gemini/config/mcp_config.json <<EOF
{
  "mcpServers": {
    "campus": {
      "command": "$HOME/oficina-mcp/.venv/bin/python",
      "args": ["$HOME/oficina-mcp/server.py"]
    }
  }
}
EOF
```

Abrir fora do projeto

cd ~/chat-campus e agy, para o agente não ler o server.py em vez de chamar a tool

Conferir com /mcp

O servidor campus aparece conectado, com as duas tools

Sinal de sucesso

`Called 1 MCP tool (campus/cardapio_do_dia)`

Conectando ao Claude Desktop

```
{
  "mcpServers": {
    "campus": {
      "command": "wsl.exe",
      "args": ["-e", "bash", "-c",
        "$HOME/oficina-mcp/.venv/bin/python $HOME/oficina-mcp/server.py"]
    }
  }
}
```

Arquivo certo

Na versão MSIX, o app lê o arquivo dentro de %LOCALAPPDATA%\Packages; o botão Editar configuração abre outro.

Reiniciar de verdade

Sair pelo ícone da bandeja: a configuração só é lida ao iniciar.

Conferir

Configurações → Desenvolvedor: campus “Em execução”

Servidores MCP para testar

Servidor	Onde roda	Login	O que faz
Everything	Local (npx)	Não	Servidor de teste com tools, resources e prompts
time (oficial)	Local (pip)	Não	Hora atual e conversão de fusos horários
DeepWiki	Remoto	Não	Perguntas sobre repositórios públicos do GitHub
Microsoft Learn	Remoto	Não	Busca na documentação oficial da Microsoft
AWS Knowledge	Remoto	Não	Documentação da AWS e disponibilidade por região
GitHub, Notion, Drive	Remoto	Sim (OAuth)	Conectores que acessam os seus dados

Catálogos: MCP Registry (oficial, em prévia) e mcpservers.org (comunidade, sem verificação dos servidores).

Resumo e referências

- O MCP padroniza a conexão entre aplicações de IA e ferramentas ou dados externos.
- Servidores expõem tools, resources e prompts; o cliente os acessa por JSON-RPC.
- O modelo escolhe a ferramenta; o servidor executa, sob controle do host.

REFERÊNCIAS

Anthropic Academy, curso “Introduction to Model Context Protocol”: academy.claude.com

Documentação oficial do protocolo: modelcontextprotocol.io

SDK oficial para Python: py.sdk.modelcontextprotocol.io

Roteiro e códigos

github.com/lfserique/meu-mcp